Problem Statement: Find the Missing Number in an Array

- Description: Given an array containing n distinct numbers taken from 0, 1, 2, ..., n, find the one that is missing from the array.
- Example:
  - Input: [3, 0, 1]
  - Output: 2 (The missing number is 2)

1. Using Loops and conditional statements

Given an array containing `n` distinct numbers taken from `0, 1, 2, ..., n`, find the one that is missing from the array.

Approach:

1. Iterate through all numbers from `0` to `n`.
2. For each number, check if it is present in the array using a nested loop.
3. If a number is not found in the array, that number is the missing one.

C# Implementation

C#

```
using System;

public class Solution
{
    public int MissingNumber(int[] nums)
    {
        int n = nums.Length;

        // Iterate through all numbers from 0 to n
        for (int i = 0; i <= n; i++)
        {
            bool found = false;

            // Check if the current number i is present in the array
            for (int j = 0; j < n; j++)
            {
                if (nums[j] == i)
                {
```

```
                found = true;
                break;
            }
        }

        // If the number i is not found in the array, return it as the missing
number
        if (!found)
        {
            return i;
        }
    }

    // This line should never be reached if the input array is valid
    return -1;
  }

  public static void Main(string[] args)
  {
    Solution solution = new Solution();
    int[] nums = { 3, 0, 1 };
    int result = solution.MissingNumber(nums);
    Console.WriteLine("The missing number is: " + result); // Output: The
missing number is 2
  }
}
```

Explanation of the Code:

1. **Initialization**:
    - Get the length of the array `n`.
2. **Outer Loop**:
    - Iterate through all numbers from `0` to `n`.
3. **Inner Loop**:
    - For each number `i`, use a nested loop to check if the number `i` is present in the array.
4. **Check Presence**:
    - If the number `i` is found in the array, set `found` to `true` and break out of the inner loop.

    - If the number `i` is not found after checking all elements of the array, return `i` as the missing number.
5. **Edge Case**:
    - The return statement outside the loops is a fallback, but it should never be reached if the input array is valid and the problem constraints are met.

Complexity Analysis:

- **Time Complexity**: O(n^2), where `n` is the length of the array. This is because we have a nested loop where both the outer and inner loops iterate up to `n`.
- **Space Complexity**: O(1), as we are not using any extra space beyond the variables for iteration and checking.

This approach is straightforward and uses simple loops and conditions to find the missing number without relying on mathematical formulas or bit manipulation techniques.

2. Using Mathematical formula

To solve the problem of finding the missing number in an array containing distinct numbers from 0 to `n`, we can use a mathematical approach based on the sum of the first `n` natural numbers. The sum of the first `n` natural numbers is given by the formula:

$$\text{Sum} = n \times (n+1)/2$$

By calculating the actual sum of the elements in the array and subtracting it from the expected sum, we can find the missing number. This approach has a time complexity of O(n) and a space complexity of O(1).

C# Implementation

C#
```
using System;
```

```
public class Solution
{
    public int MissingNumber(int[] nums)
    {
        int n = nums.Length;
        int expectedSum = n * (n + 1) / 2;
        int actualSum = 0;

        foreach (int num in nums)
        {
            actualSum += num;
        }

        return expectedSum - actualSum;
    }

    public static void Main(string[] args)
    {
        Solution solution = new Solution();
        int[] nums = { 3, 0, 1 };
        int result = solution.MissingNumber(nums);
        Console.WriteLine("The missing number is: " + result); // Output: The
missing number is 2
    }
}
```

Explanation of the Code:

1. **Calculate the Expected Sum**:
    - Using the formula for the sum of the first `n` natural numbers: `n * (n + 1) / 2`.
2. **Calculate the Actual Sum**:
    - Iterate through the array and sum up all the elements.
3. **Find the Missing Number**:
    - The missing number is the difference between the expected sum and the actual sum.

Complexity Analysis:

- **Time Complexity**: O(n), where `n` is the length of the array. This is because we iterate through the array once to compute the actual sum.
- **Space Complexity**: O(1), as we are using a fixed amount of extra space (for variables `expectedSum` and `actualSum`).

Example Walkthrough:

For the input array `[3, 0, 1]`:

1. `n = 3` (length of the array).
2. Expected Sum = `3 * (3 + 1) / 2 = 6`.
3. Actual Sum = `3 + 0 + 1 = 4`.
4. Missing Number = `6 - 4 = 2`.

Thus, the missing number is `2`.

This solution is efficient and leverages basic arithmetic to find the missing number in linear time with constant space.